

ADVENTIST UNIVERSITY OF CENTRAL AFRICA

Distributed Multi-Model Analytics for E-commerce Data

AUCA Big Data Analytics Final Project: Technical Report

Student Name	Student ID
Felicien RWAGASANA	101364

Program: MSc in Big Data Analytics
Course: Advanced Big Data Analytics
Lecturer: Temitope Oguntade
GitHub: RwagasanaFelc/BIG-DATA-ANALYTICS-FINAL-PROJECT
sDate: November 2026

Contents

1. System Architecture Overview 3
2. Data Modeling Decisions and Rationale (Part 1) 3
    2.1 MongoDB document model 3
    2.2 MongoDB aggregation pipelines 4
    2.3 HBase wide-column model 4
    2.4 Why some data is in MongoDB and some in HBase 4
3. Spark Processing Pipelines (Part 2) 5
    3.1 Batch cleaning and recommendation indicator 5
    3.2 Optimisation choices 5
    3.3 Spark SQL analytics 5
4. Integrated Analytics Workflow (Part 3) 6
5. Scalability Considerations 6
6. Technology Selection Justification 7
7. Key Findings and Business Insights (Part 4) 7
8. Visualization of Results 8
9. Methodology 9
10. Limitations and Future Work 9

1. System Architecture Overview

This project analyses a synthetic e-commerce dataset with three systems. The data comes from the generator supplied with the assignment: 10,000 users, 25 categories, 5,000 products, 500,000 transactions and 20 session files of 100,000 sessions each. That is about 2 million sessions and 3.8 GB of JSON, covering August to early November 2026. I divided the work according to what each system does well. MongoDB holds the catalogue, the users, the transactions and a subset of the sessions, and is used for aggregation queries. HBase holds session data keyed by user and time, plus daily product counters. Spark reads the raw files for cleaning and SQL analysis and joins results from the two databases in the integrated query.

Component	Technology	Role	Data held
Document store	MongoDB 8.0 (Docker)	Aggregation queries on nested documents	All users, categories, products and transactions; 200,000 sessions
Wide-column store	HBase (Docker, Thrift on port 9090)	Per-user session retrieval and daily product counters	50,000 sessions; 141,685 daily product rows
Processing	PySpark 4.3.0, Java 17, local mode	Cleaning, recommendations, Spark SQL, integrated CLV query	Raw JSON files; aggregates from MongoDB and HBase
Visualization	Python, matplotlib	Charts used in this report	Query results

Table 1. Components of the system

Everything ran on one Windows laptop. MongoDB and HBase run as Docker containers and Spark runs in local mode. Python loaders write to MongoDB with pymongo and to HBase with happybase through the Thrift port. The integrated query reads aggregates from both databases and joins them in Spark, and the results are plotted with matplotlib.

2. Data Modeling Decisions and Rationale (Part 1)

2.1 MongoDB document model

I kept data that is read together inside the same document. Each category document contains its subcategories, since the two levels are always used together. Product documents carry the category name, the subcategory name and the subcategory profit margin, copied from the categories file when loading. This duplicates a little data, but revenue and popularity queries need the category name all the time and category names almost never change, so I preferred this to a lookup in every query. The price history stays as an array inside the product. User documents keep the profile and the nested `geo_data`, with the registration and last-active timestamps stored as dates instead of strings. Transactions embed their line items, because a transaction means little without them. Each session is one document with the device profile, referrer, cart and the page_views array, so a whole journey can be read in one go.

I created unique indexes on `user_id` and `product_id`, an index on the category id in products, indexes on `user_id`, `timestamp` and `items.product_id` in transactions, and a compound index on `user_id` and `start_time` in sessions. These match the filters and joins used by the queries below. The loader (`load_mongo.py`) inserts in batches of 5,000. Users, products, categories and all 500,000 transactions were loaded; for sessions I loaded two of the twenty files (200,000 sessions) because of their size.

2.2 MongoDB aggregation pipelines

The first pipeline calculates revenue by category. It unwinds the items array of each transaction, looks up the product to get its category, groups by category name while summing item subtotals and quantities, and keeps the top ten. Cameron-Parsons came first with 22.2 million from 80,261 units, and Bush Inc was tenth with 19.7 million, so the gap between first and tenth place is only about 12 percent.

The second pipeline segments users by purchase frequency. Transactions are grouped by user to get an order count and a total spend, and `$bucketAuto` then cuts the users into four groups of roughly equal size. Average spend goes up with purchase frequency (Table 2).

Purchases per user	Users	Average total spend
27 to 46	2,666	38,671
46 to 51	2,708	44,972
51 to 56	2,510	49,299
56 to 80	2,116	55,744

Table 2. Users segmented by number of purchases

2.3 HBase wide-column model

HBase sorts rows by key, so the design is mostly about choosing the key. For `user_sessions` I used `user_id#reverse_timestamp#session_id`, where the reverse timestamp is 9999999999 minus the session start time in epoch seconds, padded to ten digits. All sessions of one user then sit next to each other with the newest first, and a prefix scan with a limit returns recent activity without any sorting. The table has three column families. `meta` holds the session id, start and end time, duration, conversion status, referrer and location. `device` holds the device type, operating system and browser. `views` holds the viewed products, the cart contents and one column per page view (`pv_000`, `pv_001` and so on). Sessions have different numbers of page views and HBase only stores the cells that exist, which suits this kind of sparse event data. Keeping the page views in their own family also means that a query on the other columns does not have to read them.

The second table, `product_metrics`, uses the key `product_id#YYYYMMDD` with one family `m` holding the counters `views`, `cart_adds` and `purchases`. A prefix scan on a product returns its daily series in date order, and a start and stop row gives any date range.

I loaded 50,000 sessions from the first session file. This produced 50,000 rows in `user_sessions` and 141,685 rows in `product_metrics`, since the daily counters are computed in the loading script. I then ran three queries with happybase: all sessions of `user_000000` by prefix scan, which returned four sessions from the newest (28 October) to the oldest (13 August 2026); the three latest sessions of the same user using a limit and a column filter; and seven days of views, cart additions and purchases for one product. The shell equivalents of the schema are `create 'user_sessions', 'meta', 'device', 'views'` and `create 'product_metrics', 'm'`; they are also in `create_hbase_tables.py`.

2.4 Why some data is in MongoDB and some in HBase

The split follows the way the data is queried. Catalogue, user and transaction data are analysed in many different ways (by category, by user, by month), which needs a flexible query language, so MongoDB is the better fit. Session events are almost always accessed through a user and a time range, and there are far more of them, which is the case HBase is designed for.

Criterion	MongoDB	HBase
Access pattern	Ad hoc aggregations, grouping and joins across collections	Lookups and range scans on a known row key
Data shape	Nested documents and arrays (items, page views)	Sparse wide rows with many optional columns, ordered by time
Used here for	Catalogue, users, transactions, revenue and segmentation	Per-user session streams and daily product counters
Main weakness	Cross-collection joins and very large arrays get expensive	Every query has to fit the row key design

Table 3. MongoDB and HBase compared for this project

3. Spark Processing Pipelines (Part 2)

3.1 Batch cleaning and recommendation indicator

`spark_batch.py` reads `transactions.json` with the `multiLine` option because the file is one JSON array. The cleaning step removes duplicate transaction ids, drops rows without a user or without items, converts the timestamp, turns empty session ids into nulls (a purchase does not always have a session), sets missing discounts to zero and trims and lowercases the status. All 500,000 rows came through unchanged, because the generated data is already consistent, but I kept the rules so the job would still work on dirtier input. I cleaned only the transactions file, which is the one used for the recommendation and revenue analyses.

For the recommendation indicator I looked at products bought together. The items are exploded into pairs of transaction and product, made distinct, and joined to themselves on the transaction id with the condition that the first product id is smaller than the second, so each pair is counted once. The most frequent pair (prod_02483 and prod_02898) appears in only five transactions and the next five pairs in four. The generator builds baskets at random, so there is little real signal here, but the method would work on real baskets.

3.2 Optimisation choices

I ran Spark in local mode with 4 GB of driver memory and lowered the number of shuffle partitions from 200 to 16, because the default creates far too many small tasks on a single machine. Columns are selected before the joins, distinct is applied before the self-join to avoid double counting, and the smaller-than condition halves the number of pairs. Only small aggregates are brought back to the driver. On a cluster I would also broadcast small tables such as products and store the data as Parquet partitioned by date instead of JSON. I considered the Spark connectors for MongoDB and HBase but did not use them, because setting them up on Windows would have taken more time than I had. The integrated query therefore passes aggregated results instead.

3.3 Spark SQL analytics

`spark_sql.py` registers the JSON files as temporary views and runs four queries. The first computes monthly revenue and month-on-month growth with the `LAG` window function. The second ranks products within each category by revenue with `RANK()` over a partition, after joining the exploded line items to the products. The third is a cohort analysis: users are grouped by registration month, and I count buyers and revenue by the number of months since registration. The fourth computes the conversion rate by device type on one session file, which stands in for

data that would come from HBase. The queries are in the repository and their output is in `spark_sql_output.txt`. Table 4 shows the monthly revenue result.

Month	Orders	Revenue (millions)	Growth on previous month
2026-08	123,207	115.0	-
2026-09	166,596	155.5	+35.2%
2026-10	172,860	161.0	+3.5%
2026-11 (to 7 Nov)	37,337	35.0	partial month

Table 4. Monthly revenue (Spark SQL query 1)

The cohort query shows the same number of buyers in every month of a cohort (the February cohort has 1,019 buyers each month), so no customer ever drops out. This comes from the generator and is not something I would expect in real data. Conversion by device is almost identical: 20.33% on desktop, 20.23% on tablet and 20.16% on mobile.

4. Integrated Analytics Workflow (Part 3)

The business question is which customers are the most valuable and whether they behave differently on the site. I estimated customer lifetime value (CLV) from purchases and added engagement data from the sessions (`integrated_clv.py`).

The purchase side comes from MongoDB. One aggregation groups the 500,000 transactions by user to get the number of orders and the total spend, and a second finds the dates of the first and last transaction, so only 10,000 small rows leave the database. The engagement side comes from HBase: a scan of `user_sessions` that requests only the duration and conversion status columns, from which I count sessions, total duration and converted sessions per user. The 50,000 sessions cover all 10,000 users, about five each. In Spark the two result sets become DataFrames and are joined on `user_id` with a left join. Annual CLV is the total spend divided by the number of days between the first and the last transaction, multiplied by 365, and `NTILE(4)` over this value splits the users into four quartiles. Table 5 shows the engagement in each quartile.

CLV quartile	Users	Average annual CLV	Sessions per user	Session conversion
1 (lowest)	2,500	148,910	4.9	18.1%
2	2,500	177,672	5.0	19.8%
3	2,500	198,955	5.0	21.3%
4 (highest)	2,500	231,354	5.2	22.1%

Table 5. CLV quartiles and engagement

Each system does the part it handles best. MongoDB does the heavy grouping next to the data, HBase serves the per-user session records through a narrow column scan, and Spark does the join and the window function, which is the step that has to scale if both sources grow. Pushing the aggregation into MongoDB and selecting only two columns in HBase keeps the amount of data sent to Spark small. With much larger data the same workflow would use the connectors and parallel HBase scans instead of a single client scan.

5. Scalability Considerations

For MongoDB I would shard transactions and sessions on `user_id` or a hashed key so that the load is spread across shards, and keep the category data inside the products so that revenue queries do not need cross-shard lookups. Summary collections written with `$merge` would avoid recomputing the same aggregations for dashboards.

In HBase the user_id prefix spreads writes over regions and the reverse timestamp keeps the most recent rows together. If ingestion became very heavy I would pre-split the regions and add a salt to the keys if hotspots appeared.

For Spark the first change would be running on a cluster instead of local mode. A JSON array in one file cannot be split across workers, so the data should be stored as JSON Lines or, better, as Parquet partitioned by date. Small tables should be broadcast in joins, and the number of shuffle partitions should follow the data size. Processing all 3.8 GB of session JSON on one laptop is not practical, which is why I worked with subsets.

6. Technology Selection Justification

Table 6 shows which tool I used for each type of analysis and the reason.

Analytical task	Technology	Reason
Revenue, popularity and segmentation	MongoDB aggregation	Items and category data are nested in the documents, and the pipelines run next to the data
Recent sessions of a user, daily product series	HBase	Row key prefix scans over sparse events ordered by time
Cleaning, co-purchase, cohorts and window functions	Spark and Spark SQL	Distributed joins and window functions over all the files
Customer lifetime value	MongoDB, HBase and Spark together	Each source is queried where it is strongest and the results are joined in Spark

Table 6. Technology chosen for each analytical task

7. Key Findings and Business Insights (Part 4)

The clearest finding is in the conversion funnel (Figure 4). Almost every session views a product (98.7%) and 65.0% add something to the cart, but only 20.2% end in a purchase. About 69% of the sessions with a cart do not convert, so the step from cart to purchase is where the most revenue is lost. Cart reminders or a simpler checkout would be the first things to test.

Revenue grew by 35.2% from August to September and by 3.5% in October (Figure 1). The fall shown for November is not a real drop, since the data stops on 7 November. The ten largest categories earn between 19.7 and 22.2 million (Figure 2), so no category dominates and promotions do not need to concentrate on a few of them.

The customer value analysis (Figure 3) shows that the most valuable quartile converts 22.1% of its sessions, against 18.1% for the least valuable, while the number of sessions per user is almost the same (5.2 against 4.9). High-value customers therefore stand out by converting better and not by spending more time on the site, so raising the conversion of lower-value customers looks like the most promising lever. Device type has no visible effect on conversion in this dataset. Many of these differences are small because the data is random, as discussed in Section 10.

8. Visualization of Results

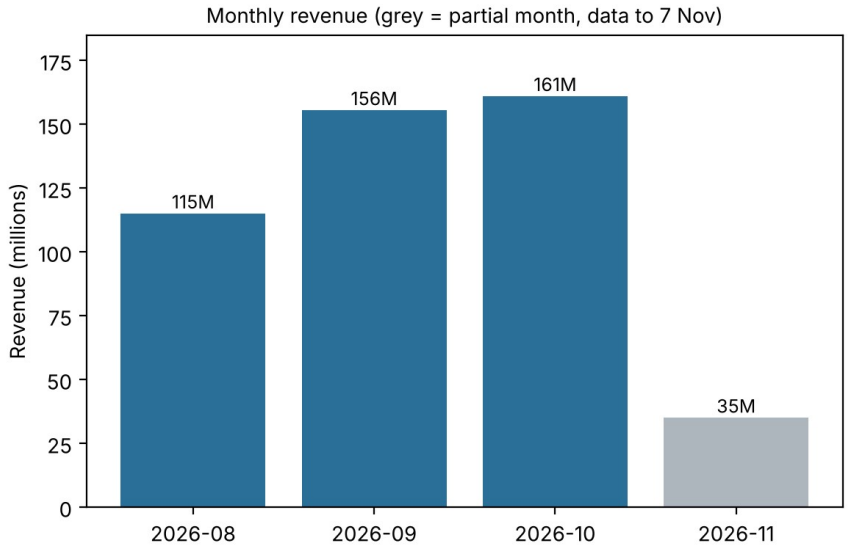


Figure 1. Monthly revenue from MongoDB (grey bar: partial month, data until 7 November).

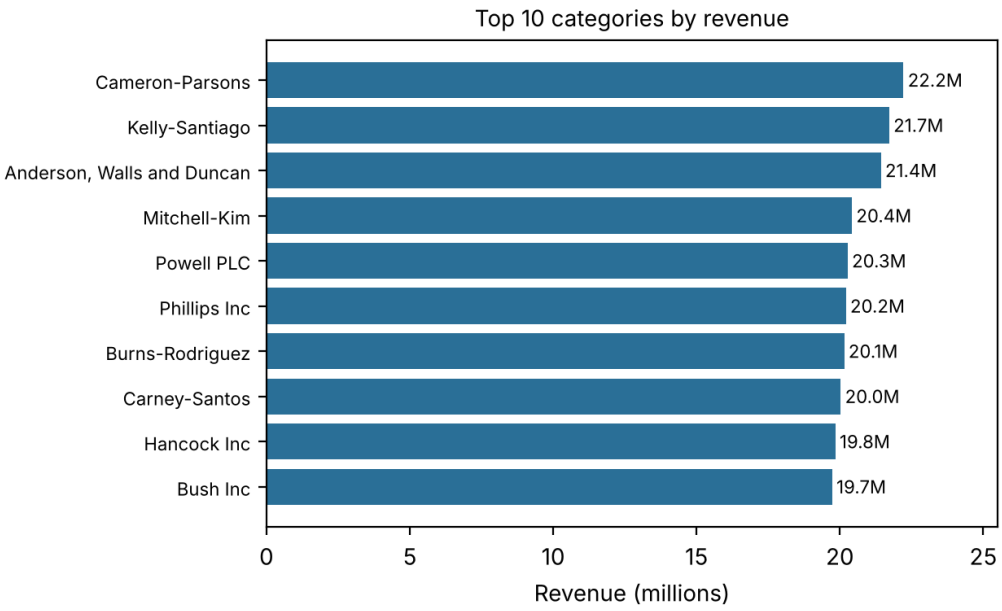


Figure 2. Top 10 categories by revenue (MongoDB aggregation pipeline 1).

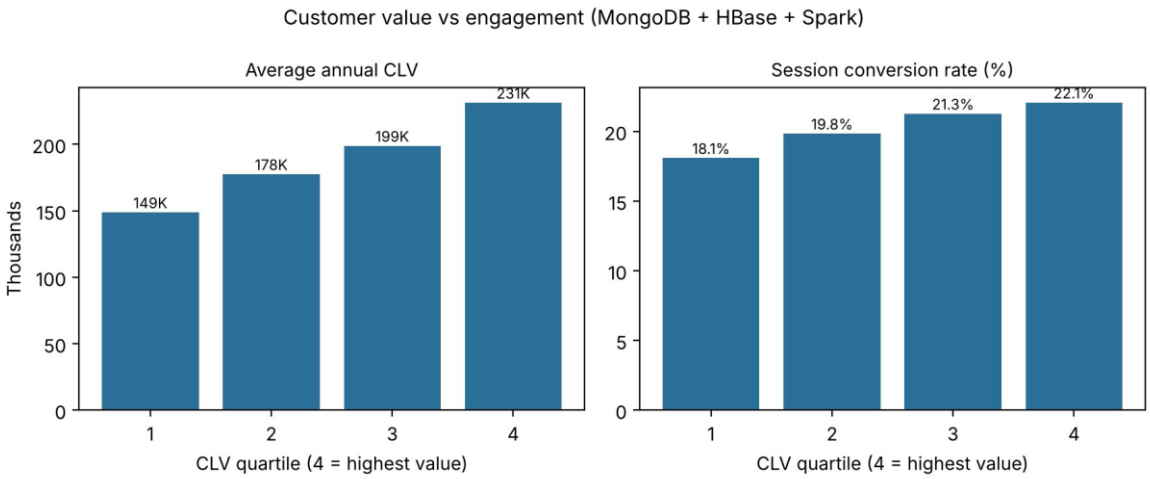


Figure 3. Customer value quartiles compared with session conversion (integrated MongoDB, HBase and Spark query).

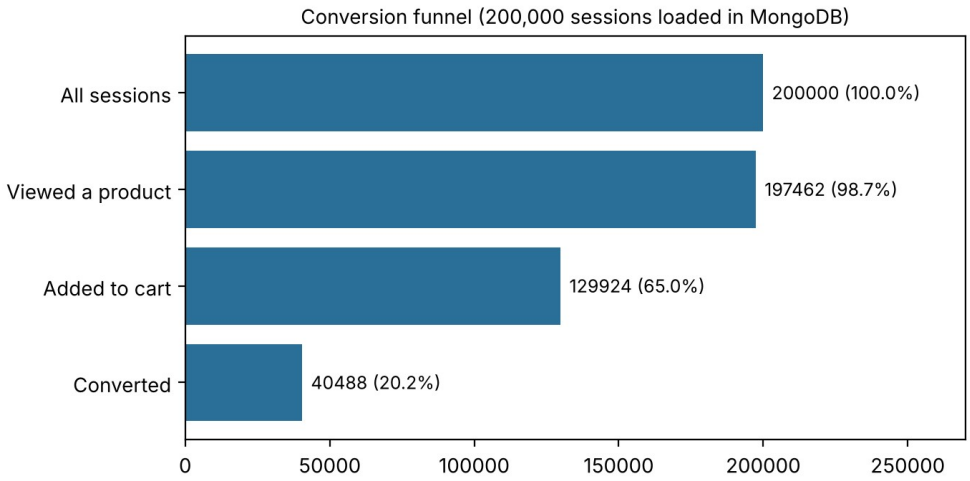


Figure 4. Conversion funnel for the 200,000 sessions stored in MongoDB.

9. Methodology

Revenue is the sum of the transaction totals over all statuses (completed, delivered, shipped and processing); there are no cancelled orders in the data. In the funnel, a session counts as having viewed a product if it has at least one `product_detail` page view, as having added to the cart if its cart is not empty, and as converted if its conversion status is converted. Customers were segmented into equal-sized groups, with `$bucketAuto` for purchase frequency and `NTILE(4)` for CLV. Cohorts are defined by registration month. CLV is a simple annualisation of the observed spend, without discounting and without a churn model. All steps can be reproduced with the scripts in the repository by following the README.

10. Limitations and Future Work

The data is synthetic and randomly generated, so several results are almost flat: category revenue, conversion by device and customer retention. They show that the pipelines work, but I would not draw business conclusions from them as they stand. MongoDB holds 200,000 and HBase 50,000

of the 2 million sessions, the device conversion query used a single session file, and Spark cleaning was applied to the transactions file only. November contains just one week of data.

On the technical side, HBase ran as a single-node Docker container accessed through the Thrift gateway. At first the gateway accepted connections but did not answer requests, and the problem went away after I restarted the container. Spark did not read from the databases through native connectors, and the user documents in MongoDB do not contain a purchase summary, which is computed in the integrated query instead. CLV is a simple extrapolation and not a predictive model.

With more time I would add the Spark connectors for MongoDB and HBase, simulate a live stream of sessions with Spark Structured Streaming, build product recommendations with collaborative filtering (ALS), store precomputed purchase summaries in the user documents, and schedule the whole pipeline.